Streaming Replication

PostgreSQL 18 Master on Ubuntu Host → Replica in Docker (Debian Slim)

1. Host (Master) Setup — Ubuntu

1.1 Install PostgreSQL 18

```
sudo apt update
sudo apt install -y wget gnupg lsb-release

# Add PostgreSQL repo
echo "deb http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" \
    | sudo tee /etc/apt/sources.list.d/pgdg.list
wget -qO - https://www.postgresql.org/media/keys/ACCC4CF8.asc \
    | sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/postgresql.gpg

sudo apt update
sudo apt install -y postgresql-18 postgresql-client-18
```

1.2 Conf ġure PostgreSQL for replication

Edit /etc/postgresql/18/main/postgresql.conf:

```
listen_addresses = '*'        # allow external connections
wal_level = replica
max_wal_senders = 10
wal_keep_size = 64MB
hot_standby = on
```

Edit /etc/postgresql/18/main/pg_hba.conf to allow replication:

```
Replication users from replica container
host replication slave1 172.17.0.2/32 scram-sha-256          #use docker container
host replication slave2 172.17.0.3/32 scram-sha-256
```

Create replication users:

```
sudo -u postgres psql
CREATE USER slave1 REPLICATION LOGIN PASSWORD 'slave1';
CREATE USER slave2 REPLICATION LOGIN PASSWORD 'slave2';
```

Restart PostgreSQL:

```
sudo systemctl restart postgresql
sudo systemctl status postgresql
```

Verify listening:

```
ss -lntp | grep 5432
```

2. Docker Container (Replica) — Debian Slim

2.1 Create Persistent Debian Container

Create a volume for persistence

```
docker volume create debian_slim_data
```

Run container interactively with persistent storage

```
docker run -it \
  --name pg18-replica \
  --network bridge \
  --mount source=debian_slim_data,target=/root \
  debian:bookworm-slim bash
```

Inside the container:

```
apt update
apt install -y wget gnupg lsb-release locales sudo nano curl
locale-gen en_US.UTF-8
update-locale LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

2.2 Install PostgreSQL 18 in Container

```
echo "deb http://apt.postgresql.org/pub/repos/apt bookworm-pgdg main" > /etc/apt/s
wget -qO - https://www.postgresql.org/media/keys/ACCC4CF8.asc | gpg --dearmor -o /
apt update
```

```
apt install -y postgresql-18 postgresql-client-18
```

2.3 Prepare Replica Data Directory

```
pg_ctlcluster 18 main stop
rm -rf /var/lib/postgresql/18/main/*
chown -R postgres:postgres /var/lib/postgresql/18/main
chmod 700 /var/lib/postgresql/18/main
```

2.4 Base Backup from Master

```
su - postgres
pg_basebackup -h <MASTER_IP> -D /var/lib/postgresql/18/main -U slave1 -Fp -Xs -P -
```

2.5 Start Replica Cluster

```
pg_ctlcluster 18 main start
pg_ctlcluster 18 main status
psql -c "SELECT pg_is_in_recovery();" # Should return 't'
```

2.6 Troubleshooting

Locale issues:

```
apt install -y locales
locale-gen en_US.UTF-8
update-locale LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

Permission denied:

```
chown -R postgres:postgres /var/lib/postgresql/18/main
chmod 700 /var/lib/postgresql/18/main
```

Replication access: ensure correct `pg_hba.conf` entry for container IP.

Network connectivity:

```
telnet <MASTER_IP> 5432
```

3. Verify Replication

On master:

```
sudo -u postgres psql
SELECT application_name, client_addr, state, sync_state, write_lag, flush_lag, rep
```

On replica:

```
psql -c "SELECT pg_is_in_recovery();"
```

4. Notes / Best Practices

- Keep PostgreSQL data directory persistent: map `/var/lib/postgresql/18/main` to Docker volume.
- Use a bridge network in Docker for controlled IPs.
- Always stop cluster before base backup.
- Match locales between host and container.
- For multiple replicas, repeat base backup with separate replication users.

5. Full Workf bw Summary

1. **Master Preparation (Ubuntu Host)**
   - Install PostgreSQL 18 via PGDG repository.
   - Conf ġure `postgresql.conf` for replication (`wal_level`, `max_wal_senders`, `wal_keep_size`, `hot_standby`).
   - Update `pg_hba.conf` to allow replication users from Docker container IPs.
   - Create replication users with `REPLICATION LOGIN`.
   - Restart PostgreSQL and verify it listens on 5432.
2. **Docker Replica Setup (Debian Slim)**
   - Create persistent Docker volume.
   - Run Debian container and conf ġure locales (`en_US.UTF-8`).
   - Install PostgreSQL 18 via PGDG repository.
   - Stop default cluster, clear `/var/lib/postgresql/18/main`, set proper ownership and permissions.
3. **Replication Setup**

- Use `pg_basebackup` from the master to initialize replica.
- `-R` flag creates standby configuration automatically.
- Start PostgreSQL cluster on replica.
- Verify replication mode with `SELECT pg_is_in_recovery();`.

4. **Validation**
   - On master: check `pg_stat_replication`.
   - On replica: ensure `pg_is_in_recovery()` returns `t`.
   - Test network connectivity between container and master.
5. **Maintenance / Troubleshooting**
   - Ensure locales match between master and replica.
   - Check permissions on data directory.
   - Verify `pg_hba.conf` entries for replication.
   - Monitor WAL lag and replication status.
   - Promote replica if needed and adjust configurations to prevent split-brain.
6. **Best Practices**
   - Map data directory to persistent Docker volume.
   - Use controlled Docker bridge network with static IPs.
   - Use strong passwords and consider SSL for replication.
   - Automate PostgreSQL startup inside container.
   - For multiple replicas, repeat base backup with unique replication users.
   - Always stop cluster before taking base backups.